

Вопросы к собеседо**В**анию

СОДЕРЖАНИЕ

ЧАСТЬ 1. ВОПРОСЫ ПО БА.....	3
Вопрос 1. Что такое приоритизация требований и какие методы используются для расстановки приоритетов у требований?.....	3
Вопрос 2. Что такое SRS и какие ключевые элементы SRS вы знаете?	4
Вопрос 3. Что такое BRD? Чем он отличается от SRS?.....	4
Вопрос 4. Какой нотацией обычно пользуетесь при моделировании бизнес-процессов? Какой инструмент используете для моделирования?.....	5
Вопрос 5. С какими UML диаграммами работали? Какие инструменты использовали для построения UML диаграмм?.....	6
Вопрос 6. Какие бывают типы требований?	10
Вопрос 7. Какими свойствами обладают хорошие требования?.....	13
Вопрос 8. Какие существуют методы сбора требований?.....	14
Вопрос 9. Чем Use Case отличается от User Story?.....	14
Вопрос 10. Что такое методология управления проектами и какие они бывают?.....	16
ЧАСТЬ 2. ВОПРОСЫ ПО БАЗАМ ДАННЫХ (БД) И SQL.....	16
Вопрос 11. Что такое БД и какие они бывают?.....	16
Вопрос 12. Что такое ER-модель (Entity-relationship model)? Для чего нужно разрабатывать ER-модель?.....	17
Вопрос 13. В чем разница между реляционными (SQL) и нереляционными базами данных (NoSQL)?.....	17
Вопрос 14. Когда используется PRIMARY KEY?.....	18
Вопрос 15. Что такое внешний ключ?.....	18
Вопрос 16. Назовите четыре основных типа соединения в SQL.....	18
Вопрос 17. Что такое нормализация и денормализация?.....	19
Вопрос 18. Что такое индекс?.....	20
Вопрос 19. Расскажите о видах индексов.....	20
Вопрос 20. Что такое AUTO_INCREMENT?.....	21
ЧАСТЬ 3. ВОПРОСЫ ПО ИНТЕГРАЦИИ И АРХИТЕКТУРЕ.....	22
Вопрос 21. Что такое API?.....	22
Вопрос 22. В чем разница между SOAP и REST?.....	22
Вопрос 23. Что такое архитектура программного обеспечения (ПО)?.....	23
Вопрос 24. Что такое микросервисная, монолитная и клиент-сервисная архитектура? В чем разница между ними?.....	23
Вопрос 25. Какие вы знаете виды и способы интеграции систем?.....	24
ЧАСТЬ 4. ПРАКТИЧЕСКИЕ КЕЙСЫ.....	25
ЧАСТЬ 5. ТЕСТОВЫЕ ЗАДАНИЯ.....	26
P.S. от @AnakondaMar.....	28



ЧАСТЬ 1. ВОПРОСЫ ПО БИЗНЕС-АНАЛИЗУ (БА)



Вопрос 1. Что такое приоритизация требований и какие методы используются для расстановки приоритетов у требований?



Ответ: Приоритизация требований - это процесс распределения требований на основе срочности для бизнеса в зависимости от разных параметров (фазы проекта, график работ, стоимость и т.д.)

Можно выделить наиболее распространенные **методы** расстановки приоритетов:

- **RICE** (Reach — охват, Impact — влияние, Confidence — достоверность, Effort — усилия)
- **ICE** (Impact - влияние, Confidence - уверенность, Ease - легкость реализации)
- **Value vs. Effort** (Ценность против Усилия)
- **Kano**
- **MoSCoW**



Ссылки на доп.материал:

12 методов приоритизации продуктовых целей: RICE, WSJF, KANO и прочие
<https://vc.ru/marketing/274778-12-metodov-prioritizacii-produktovyh-celey-rice-wsjf-kano-i-prochie>





Вопрос 2. Что такое SRS и какие ключевые элементы SRS вы знаете?



Ответ: Спецификация системных требований (SRS) или Спецификация требований к программному обеспечению — это документ или набор документов, которые описывают функции системы или программного продукта/приложения. Документ включает в себя множество элементов, которые определяют предполагаемую функциональность, необходимую для заинтересованных сторон и заказчика для удовлетворения конечных пользователей.

Ключевые элементы SRS:

- Объем работ
- Функциональные требования
- Нефункциональные требования
- Зависимости
- Модель данных
- Предположения
- Ограничения
- Критерии приемки



Вопрос 3. Что такое BRD? Чем он отличается от SRS?



Ответ: Документ бизнес-требований (BRD) подробно описывает бизнес-решение, которое будет реализовываться в рамках проекта, в соответствии с потребностями и требованиями бизнеса/клиента. Он включает в себя цель запуска проекта, какое бизнес-решение оно предоставляет, цель выполнения проекта, функциональность решения, а также план-график его реализации.

Ключевое различие:

- **BRD - это бизнес-требования, требования заинтересованных сторон** (используется на этапе инициации/идеи), а **SRS - это функциональные/нефункциональные требования** (используется на этапе планирования).



Ссылки на доп.материал:

BRD. Как написать идеальный документ

<https://cleverics.ru/digital/2022/06/brd-kak-napisat-idealnyi-dokument/>





Вопрос 4: Какой нотацией обычно пользуетесь при моделировании бизнес-процессов? Какой инструмент используете для моделирования?



Ответ: Обычно использую **BPMN** (Business Process Modeling Notation - Нотация моделирования бизнес-процессов) для моделирования бизнес-процессов. Нотация наглядно отображает подробную последовательность бизнес-операций и информационных потоков, необходимых для завершения процесса. В нотации BPMN выделяют пять основных категорий элементов:

- элементы потока (события, процессы и шлюзы);
- данные/date (объекты данных и базы данных);
- соединяющие элементы (потоки управления, потоки сообщений и ассоциации);
- зоны ответственности (пулы и дорожки);
- Artefact (артефакты/сноски).

Шпаргалка с элементами BPMN:

	Start			Intermediate				End
	Event	sub-pr		Catch	Boundary	Throwing		
	Inter.	Non-inter.		Inter.	Non-inter.			
None								
Message								
Timer								
Error								
Escalation								
Cancel								
Compensation								
Conditional								
Link								
Signal								
Terminate								
Multiple								
Multiple parallel								



Ссылки на доп.материал:

Открытые лекции по BPMN от Дениса Котова (архитектор бизнес-процессов в Тинькофф; создатель Stormbpmn):

<https://www.youtube.com/watch?v=iFPKWpHA9pk&list=PLJHWAyPXCqb-rfiwUmEhNuPV6wIwRRscg>



Инструменты для моделирования:

- Stormbpmn (удобно, и есть проверка схемы): <https://stormbpmn.com/>
- Draw.io: <https://app.diagrams.net/>
- Camunda: <https://camundarus.ru/>

Советую подписаться на рассылку полезных материалов от Дениса Котова на сайте <https://bpmn2.ru/>

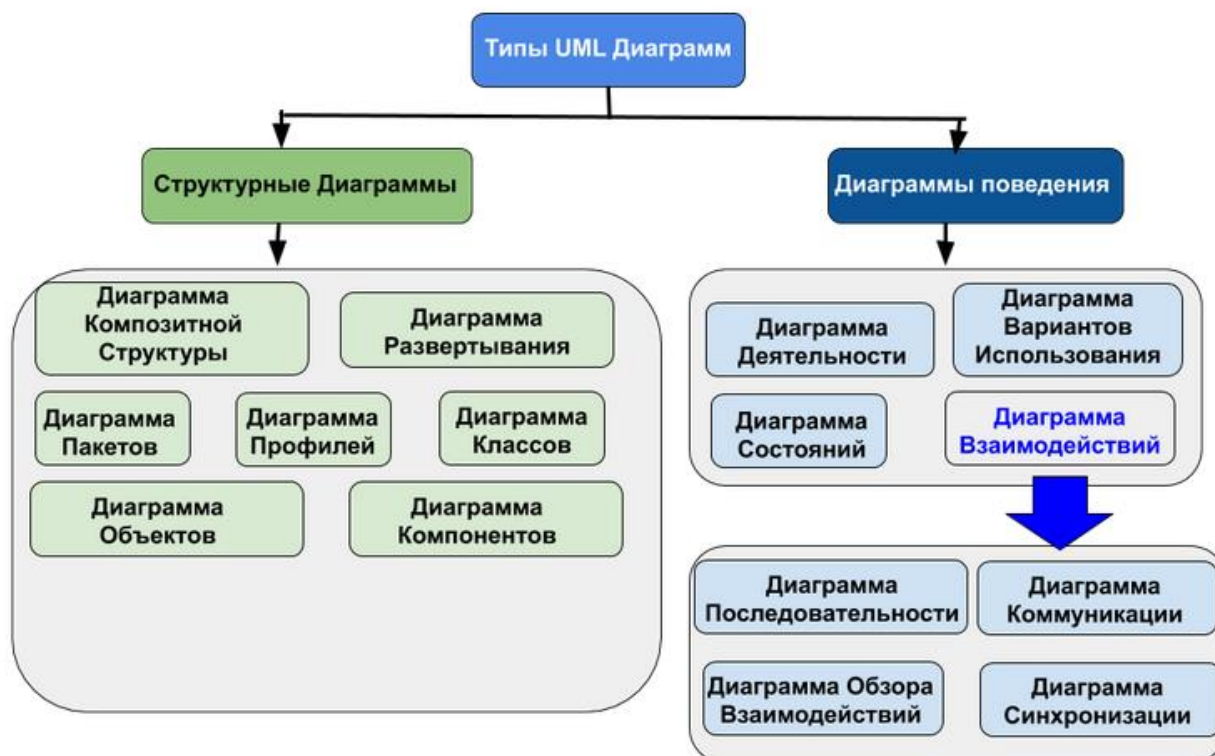
Рекомендация: обычно на собеседовании показывают 3-4 диаграммы BPMN и просят найти ошибки. Мне встречались следующие ошибки: нет завершающего события, перепутаны виды шлюзов (исключающий и параллельный), сигнал от промежуточных событий-сообщений не верно передается и т.д.



Вопрос 5. С какими UML диаграммами работали? Какие инструменты использовали для построения UML диаграмм?



Ответ: UML (Unified Modeling Language - унифицированный язык моделирования). Это графический язык, который с помощью диаграмм и схем описывает разнообразные процессы и структуры. Данный язык в основном используется для разработки программного обеспечения. Однако он также используется для описания рабочих ролей, организационных функций и бизнес-процессов.



Ниже приведу наиболее популярные диаграммы, с которыми работает БА.

Диаграмма прецедентов — Use-case diagram

Диаграмма прецедентов использует 2 основных элемента:

- 1) **Actor (участник)** — множество логически связанных ролей, исполняемых при взаимодействии с прецедентами или сущностями (система, подсистема или класс).



Участником может быть человек, роль человека в системе или другая система, подсистема или класс, которые представляют нечто вне сущности.

2) **Use case (прецедент)** — описание отдельного аспекта поведения системы с точки зрения пользователя. Прецедент не показывает, "как" достигается некоторый результат, а только "что" именно выполняется.

Пример: *product manager* у нас работает над стратегией и больше ничем не занимается, *архитектор* работает над стратегией и занимается внедрением, *build master* занимается тремя вещами одновременно, и так далее. По такой схеме мы можем проследить, какая из ролей связана с какими прецедентами.

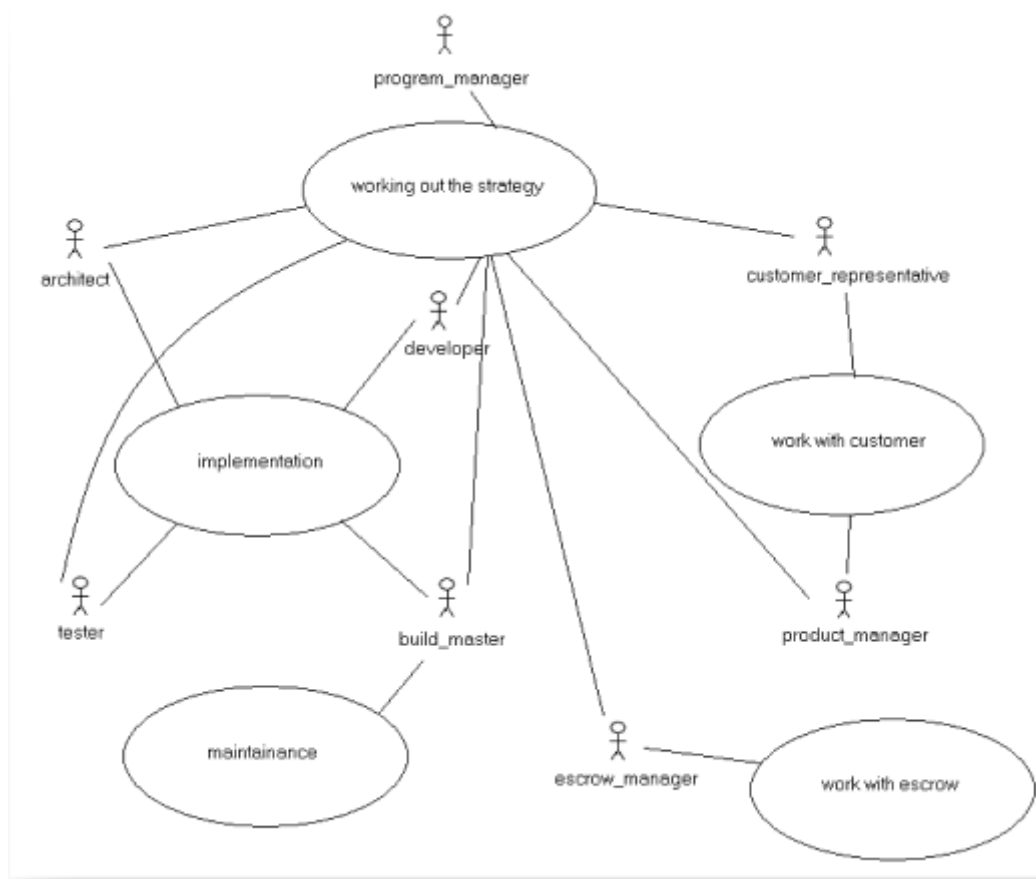
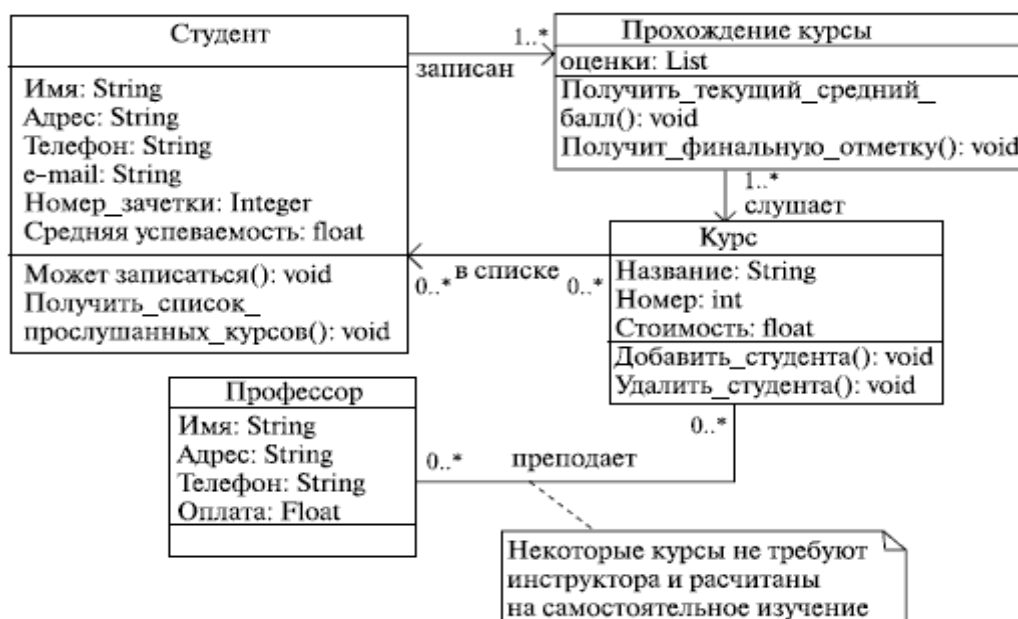


Диаграмма классов — Class diagram

Класс (class) — категория вещей, которые имеют общие атрибуты и операции. Сама диаграмма классов является набором статических, декларативных элементов модели. Она дает нам наиболее полное и развернутое представление о связях в программном коде, функциональности и информации об отдельных классах. Приложения генерируются зачастую именно с диаграммы классов. Рассмотрим на простом примере ниже:





Для класса "студент" есть таблица, содержащая атрибуты: имя, адрес, телефон, e-mail, номер зачетки, средняя успеваемость. И также показаны связи данной сущности с другими: прохождением курса, какой курс слушает, кто профессор. В этом примере также добавляются функции, которые могут быть применены к сущности "студент".

Диаграмма активностей — Activity diagram

Диаграмма активностей описывает динамические аспекты поведения системы в виде блок-схемы, которая отражает бизнес-процессы, логику процедур и потоки работ — переходы от одной деятельности к другой. По сути, мы рисуем алгоритм действий (логику поведения) системы или взаимодействия нескольких систем. Ниже — пример подобной диаграммы для интернет-магазина.



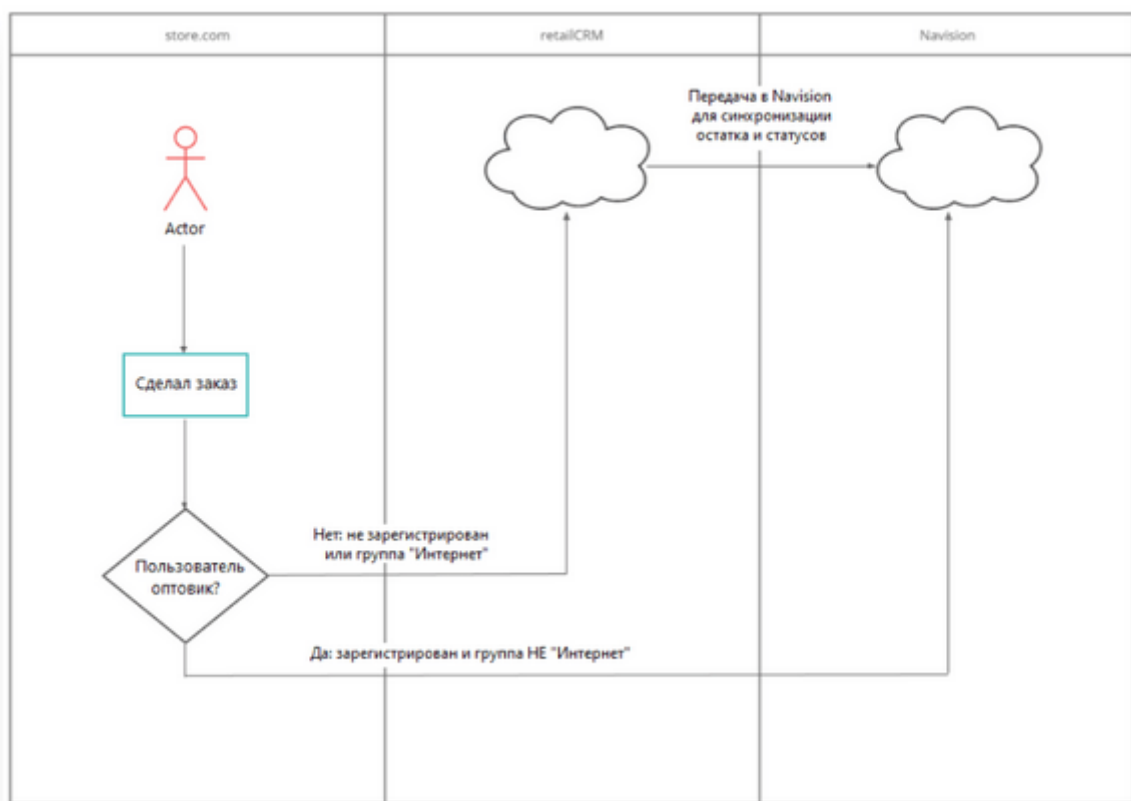
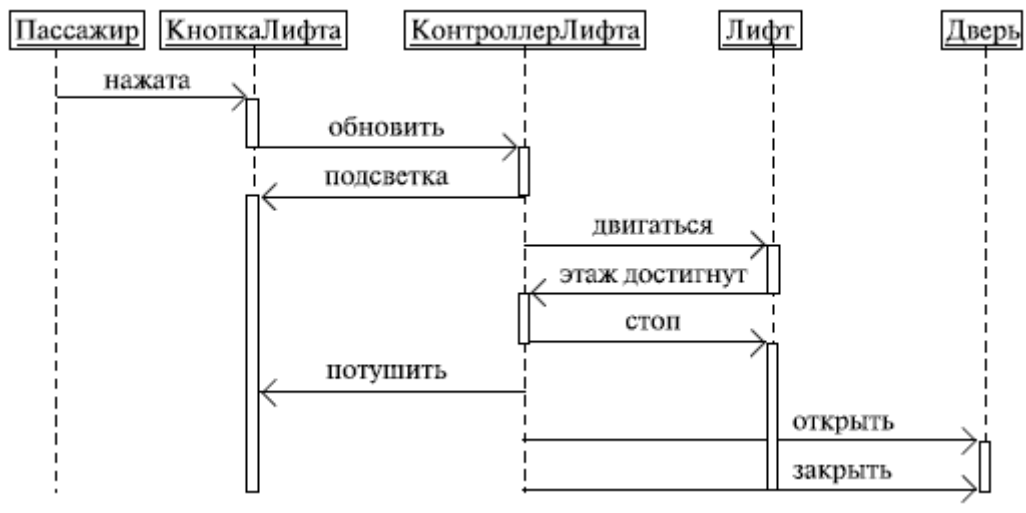


Диаграмма активностей для сайта магазина максимально доступно объясняет, какие есть интеграции в системе. Актер (в нашем случае — покупатель), зашедший на сайт, делает заказ. Далее у нас происходит разветвление: проверяем, является ли пользователь оптовиком (Да/Нет). Если он не зарегистрирован в системе и не оптовик, заказ отправляется в retailCRM. Если пользователь зарегистрирован, его заказ попадает в Navision. При этом между retailCRM и Navision происходит синхронизация остатка и статусов.

Диаграмма последовательности — Sequence Diagram

Используется для уточнения диаграмм прецедентов — описывает поведенческие аспекты системы. Диаграмма последовательности отражает взаимодействие объектов в динамике, во времени. При этом информация принимает вид сообщений, а взаимодействие объектов подразумевает обмен этими сообщениями в рамках сценария.





Ссылки на доп.материал:

Часто используемые программы для создания UML диаграмм:

- [Diagrams.net](https://diagrams.net) — удобный сервис для создания блок-схем, UML-диаграмм, моделей бизнес-процессов онлайн. Совместим с большинством популярных инструментов, включая Google Docs, Git, Dropbox, OneDrive и другие.
- [Dbdiagram.io](https://dbdiagram.io) — приложение для построения диаграмм связей для баз данных. Хороший инструмент для разработчиков и аналитиков.
- [Google Drawings](https://www.google.com/drawings/) — бесплатный инструмент для создания блок-схем и диаграмм в составе Google Drive (менее удобный по сравнению с diagrams.net);
- xmind.net — программа для построения интеллектуальных карт (mind map), логических схем, сложных структур, проведения брейншторма и не только.

Бесплатный курс по UML есть на intuit.ru.



Вопрос 6. Какие бывают типы требований?



Ответ: Типы/уровни требований:

- **Бизнес-требования (business requirements)** - высокоуровневая бизнес-цель организации или заказчиков системы
- **Пользовательские требования (user requirements)** - описывают цели или задачи, которые пользователи должны иметь возможность выполнять с помощью продукта, который в свою очередь должен приносить пользу кому-то.
- **Функциональные требования (functional requirements)** - это особенности продукта или функции, которые разработчики должны реализовать, чтобы пользователи могли выполнять свои задачи, иными словами это описание требуемого поведения системы в определенных условиях.



- **Нефункциональные требования (non-functional requirements)** - описание свойства или особенности, которым должна обладать система, или ограничение, которое должна соблюдать система.

ВАЖНО: Ключевое отличие между функциональными и нефункциональными требованиями заключается в том, что **Функциональные требования описывают, что система должна делать, а нефункциональные требования описывают, как система работает.**

Примеры функциональных требований для различных видов ПО:

Веб-сайт

Функциональные требования к веб-сайту объясняют, какие функции необходимы веб-сайту, как пользователи могут получить доступ к веб-страницам, что происходит, когда пользователи нажимают на определенные части веб-страницы, и как веб-сайт отображается в браузере. Например, возможно, клиент из торгового агентства хотел создать веб-сайт электронной коммерции, на котором клиенты могли бы узнавать о своих продуктах, отправлять запросы, просматривать запасы и размещать заказы. Некоторые из функциональных требований к веб-сайту могут включать:

- Цвет фона главной страницы светло-желтый.
- У каждого продукта есть кнопка «Добавить в корзину», которая помещает этот товар в виртуальную корзину.
- Касса безопасно обрабатывает кредитные карты всех основных поставщиков.
- Программное обеспечение соответствует всем требованиям безопасности.
- Веб-сайт позволяет администраторам компании получать доступ к данным о заказах.
- Пользователи могут щелкнуть боковую страницу навигации, чтобы просмотреть различные разделы веб-сайта.
- Контактная форма на веб-странице отправляет электронные письма непосредственно в почтовый ящик менеджера по продажам.



Мобильное приложение

Функциональные требования к мобильному приложению включают определенные функции, которые помогают пользователям перемещаться по приложению, получать доступ к ссылкам и просматривать приложение со своего мобильного устройства. Например, возможно, компания по производству одежды хотела создать мобильное приложение, позволяющее пользователям заказывать одежду через приложение. Вот некоторые потенциальные функциональные требования к приложению:

- Приложение позволяет пользователям просматривать каталоги одежды.
- Приложение требует, чтобы пользователи вводили пароль для входа в свою учетную запись.
- Пользователи могут вводить информацию о кредитной карте на странице торговой точки.
- Шаблон появляется после того, как пользователи вводят информацию о своей кредитной карте, где они могут заполнить данные о доставке.
- Программная функция в приложении позволяет пользователям автоматически добавлять товары в корзину, дважды нажав на товар.

Примеры нефункциональных требований:

Примеры нефункциональных требований

- Вот несколько примеров нефункциональных требований:
 1. **Безопасность:** Система должна быть защищена от несанкционированного доступа.
 2. **Производительность:** Система должна быть способна обслуживать необходимое количество пользователей без снижения производительности.
 3. **Масштабируемость:** Система должна иметь возможность увеличивать или уменьшать масштаб по мере необходимости.
 4. **Наличие** Система должна быть доступна, когда это необходимо.
 5. **Техническое обслуживание:** Система должна быть проста в обслуживании и обновлении.
 6. **Переносимость:** Система должна работать на разных платформах с минимальными изменениями.
 7. **Надежность:** Система должна быть надежной и соответствовать требованиям пользователя.
 8. **Юзабилити:** Система должна быть простой в использовании и понятной.
 9. **Совместимость:** Система должна быть совместима с другими системами.
 10. **Юридические вопросы:** Система должна соответствовать всем применимым законам и правилам.



Рекомендация: часто бывает так интервьюер называет то или иное требование и просит определить к какому типу оно относится (пользовательское, функциональное, нефункциональное и т.д.). Поэтому, советую посмотреть примеры разных типов требований.



Ссылки на доп.материал:

Нефункциональные требования: как не пустить систему ко дну:

<https://habr.com/ru/company/simbirsoft/blog/688428/>



Вопрос 7. Какими свойствами обладают хорошие требования?



Ответ: Хорошие требования должны удовлетворять нескольким критериям:

- Завершенность (требование должно быть описано для всех возможных условий)
- Атомарность (требование нельзя разбить на более детальные требования)
- Недвусмысленность (означает, что существует только один вариант интерпретации требования)
- Проверяемость /Тестируемость (означает, что тестеры должны иметь возможность проверить, было ли требование реализовано корректно)
- Полнота (требование должно быть описано подробно и понятно для реализации)
- Корректность (если требование содержит факты, эти факты должны быть достоверны)
- Понятность (требования должны быть грамматически правильные, написаны в соответствующем стиле)
- Независимость (чтобы понять требование, не нужно знать какое-либо другое требование) - Абстрактность (требования не должны содержать ненужной информации о дизайне и реализации системы)
- Необходимость (требование должно быть необходимым для работы системы, в требовании нет необходимости, если оно не нужно ни одному заинтересованному лицу, и удаление требования не повлияет на работу системы)
- Однозначность (требование должно быть написано понятным языком для всех, т.е. все, кто работает с требованиями, должны интерпретировать их одинаково)

Рекомендация: иногда на собеседовании просто достаточно назвать, что требования должны соответствовать критериям SMART и этого достаточно.



Ссылки на доп.материал:

Критерии качества требований:

<https://www.thebagirl.com/kharakteristiki-kachiestva-triebovanii/>





Вопрос 8. Какие существуют методы сбора требований?



Ответ: Методы сборки требований:

- **Интервью** - проходит один на один со стейкхолдером в форме свободной беседы, в офлайн или онлайн формате. Прежде чем назначить встречу необходимо подготовить тему разговора и список вопросов, и направить собеседнику, чтобы он тоже подготовился. Интервью помогает детализировать понимание глобального вопроса, разбить общую тему на отдельные задачи, уточнить требования конкретного стейкхолдера к проекту.
- **Анкетирование/опрос**
- **Фокус-группа**
- **Семинар**
- **Мозговой штурм** - метод решения задач, в котором участники обсуждения генерируют максимальное количество идей решений задачи, в том числе самые фантастические и глупые. Обычно используется для определения возможных решений проблем и уточнения деталей возможностей
- **Совещание**
- **Моделирование процессов**
- **Прототипирование** - это техника для построения быстрой и приблизительной версии желаемой системы или части этой системы. Прототип демонстрирует возможности системы пользователям и дизайнерам. Прототип представляет механизм связи, позволяющий рецензентам, понять взаимодействие внутри системы
- **Анализ вариантов использования (Use Case)** - это описательный документ, в котором излагается последовательность событий, описывающих использование пользователем системы для достижения определенных целей. Use case описывают поведение системы, предназначенное для разработки, без описания того как это поведение должно быть разработано.
- **Пользовательские истории (User Story)** это простой подход к сбору требований, который сдвигает фокус с формального документирования требований к разговору, который позволяет проекту быть более восприимчивыми с момента его создания.
- **Анализ интерфейсов**
- **Анализ действующей документации** - данная методика может быть использована при наличии в организации документации, которая может помочь в определении потребностей Заказчика. Примеры документации включают в себя: регламенты, описания процессов, структура организации, спецификации продукта, различные процедуры, стандарты и инструкции, шаблоны документов, нормативные акты и т.д.



Вопрос 9. Чем Use Case отличается от User Story?



Ответ: **User Story** помогает взглянуть на всю систему целиком.

Пользовательские истории являются результатом работы команды, в отличие от спецификаций. Обычно последние пишутся одним или несколькими людьми в команде. Как предмет коллективной работы, пользовательские истории помогают выявить все слабые стороны продукта и решить все проблемы в реализации идеи до разработки в формате живого общения.

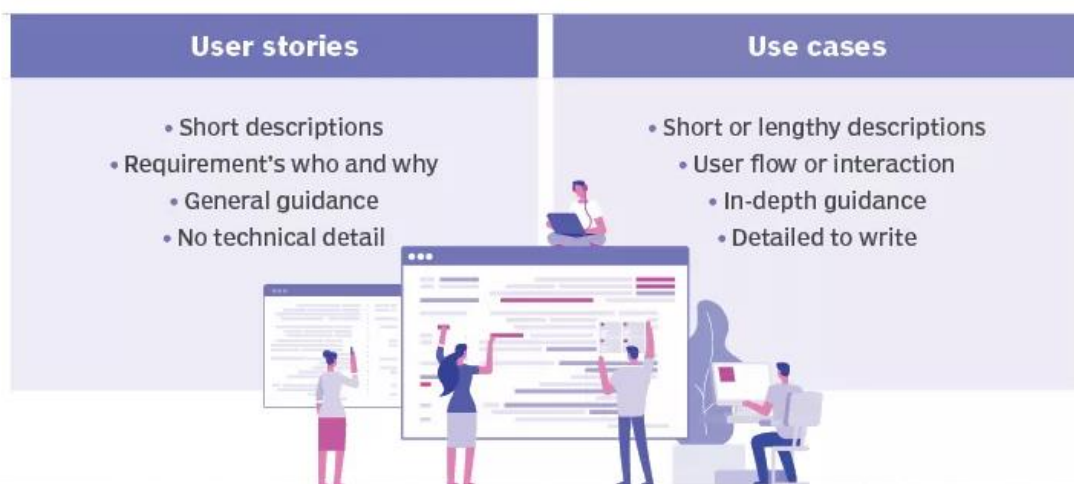


В отличие от user story, которая излагается от имени какого-то конкретного пользователя, в **use case** может быть описано взаимодействие (с определенной целью) нескольких участников. Например:

- покупка товара в магазине (Покупатель – Продавец)
- отправка письма по электронной почте (Отправитель – Почтовый клиент)
- запрос страницы браузером (браузер – веб-сервер)

User story vs. use case

Developers can—and should—rely on both user stories and use cases within Agile software development. However, the two descriptions of user needs differ.



User story

Defines the who, what and why of a product feature

Use case

Focuses on the functionalities of a feature or process



Ссылки на доп.материал:

Use case или User story? (Михаил Максимов, аналитик СБЕР)

<https://www.youtube.com/watch?v=p2hJXq1yADk>

Хороший курс по user story:

https://www.youtube.com/watch?v=WlJ83zdMdg8&list=PL8qIJB9mtdxVeu2F1Vmdie_HL0hAwVeVQ





Вопрос 10. Что такое методология управления проектами и какие они бывают?



Ответ: Методология управления проектами — это стандарт ведения проектов от старта до завершения, который включает в себя принципы работы (способы оценки сроков, постановки задач и передача их между сотрудниками, сбор требований, способы согласований и тд).

Наиболее распространенные методологии управления проектами:

- Waterfall (Водопадная модель)
- Agile (Гибкая модель)
 - - SCRUM
 - - Kanban
 - - Lean и тд.
- Гибридная модель (Waterfall+Agile)



Ссылки на доп.материал:

Методологии управления проектами: 12 популярных подходов

<https://asana.com/ru/resources/project-management-methodologies>

ЧАСТЬ 2. ВОПРОСЫ ПО БАЗАМ ДАННЫХ (БД) И SQL



Вопрос 11. Что такое БД и какие они бывают?



Ответ: База данных (БД) - это стандартный программный сервис для упорядоченного хранения данных.

Основные типы БД:

- **Реляционные** - это набор данных с predetermined связями между ними. Эти данные организованы в виде набора таблиц, состоящих из столбцов и строк.
- **Сетевые** - в отличие от реляционных баз, в сетевых между таблицами и записями, может быть несколько разных связей, каждая из которых отвечает за что-то своё.
- **Иерархические** - это сложная многокомпонентная система, в основе которой лежит древовидная структура. Она состоит из комплекса объектов разных уровней (рангов), расположенных по принципу их подчинения от общего к частному. Также есть и другие типы БД: Объектные или объектно-ориентированные, функциональные.



Ссылки на доп.материал:

Виды баз данных

<https://selectel.ru/blog/databases-types/>

11 типов современных баз данных: краткие описания, схемы и примеры БД

<https://proglab.io/p/11-tipov-sovremennyh-baz-dannyh-kratkie-opisaniya-shemy-i-primery-bd-2020-01-07>





Вопрос 12. Что такое ER-модель (Entity-relationship model)? Для чего нужно разрабатывать ER-модель?



Ответ: ER-модель или ER-диаграмма (Entity-relationship model или Entity-relationship diagram) – это семантическая модель данных, которая предназначена для упрощения процесса проектирования базы данных. Грубо говоря ER-модель – это представление базы данных в виде наглядных графических диаграмм.

В ER-моделях и моделях данных обычно выделяют до трех уровней детализации:

- **Концептуальная модель данных** - схема наивысшего уровня с минимальным количеством подробностей.

- **Логическая модель данных** - содержит более подробную информацию, нежели концептуальная модель.

- **Физическая модель данных** - на основе каждой логической модели данных можно составить одну или две физических модели.

Отвечая на вопрос для чего вообще разрабатывать ER-модель можно сказать, что с ее помощью можно выделить ключевые сущности и обозначить связи, которые могут устанавливаться между этими сущностями.



Ссылки на доп.материал:

Понятие ER-модели. Понятие сущности (entity). Атрибуты. Виды атрибутов.

<https://www.bestprog.net/ru/2019/01/24/the-concept-of-er-model-the-concept-of-essence-and-communication-attributes-attribute-types-ru/>



Вопрос 13. В чем разница между реляционными (SQL) и нереляционными базами данных (NoSQL)?



Ответ: Реляционные БД - база, где данные хранятся в формате таблиц, они строго структурированы и связаны друг с другом.

Основные СУБД реляционных БД:

-SQL: MySQL, Oracle, PostgreSQL, Microsoft SQL Server;

Нереляционная база данных (NoSQL) — хранит данные без четких связей друг с другом и четкой структуры. Вместо структурированных таблиц внутри базы находится множество разнородных документов, в том числе изображения, публикации в социальных сетях.

Основные СУБД нереляционных БД:

- NoSQL: MongoDB, Redis, RavenDB Cassandra, BigTable, HBase, Neo4j, CouchDB.

Также можно сказать, что в современном мире нет противостояния между реляционными и нереляционными БД, а лучше говорить об их совместном использовании для решения поставленных задач.



Ссылки на доп.материал:

Базы данных SQL и NoSQL: основные различия

<https://otus.ru/nest/post/2038/>





Вопрос 14. Когда используется PRIMARY KEY?



Ответ: **PRIMARY KEY** — это первичный ключ, который используется в качестве основного ключа и может быть использован для связи с дочерней таблицей, содержащей внешний ключ.



Вопрос 15. Что такое внешний ключ?



Ответ: Внешний ключ или **FOREIGN KEY** также является атрибутом ограничения и обеспечивает связь двух таблиц. По сути, это поле или несколько полей, которые ссылаются на **PRIMARY KEY** в родительской таблице.
Пример использования:

```
CREATE TABLE order (
  order_id INT NOT NULL,
  user_id INT,
  PRIMARY KEY (order_id),
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

В данном случае внешний ключ, привязанный к полю **user_id** в таблице **order**, ссылается на первичный ключ **id** в таблице **users**, и именно по этим полям происходит связывание двух таблиц.



Вопрос 16. Назовите четыре основных типа соединения в SQL.

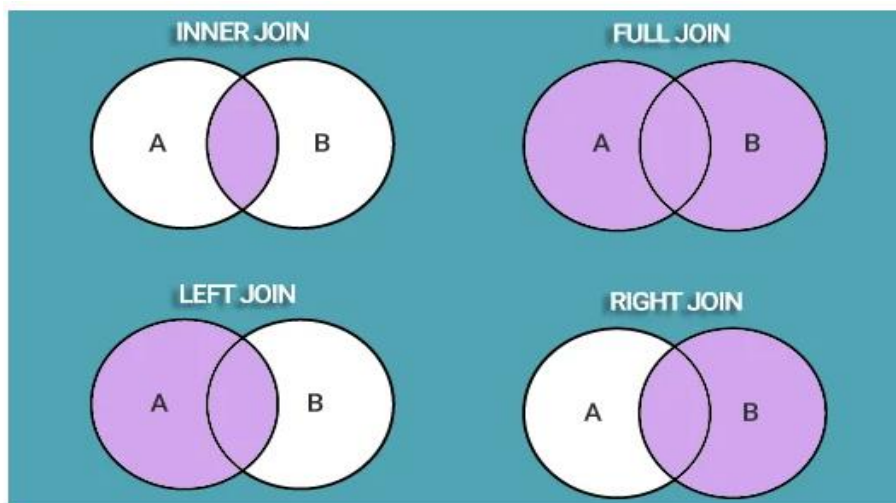


Ответ: Чтобы объединить две таблицы в одну, следует использовать оператор **JOIN**. Соединение таблиц может быть **внутренним (INNER)** или **внешним (OUTER)**, причём внешнее соединение может быть левым (LEFT), правым (RIGHT) или полным (FULL).

- **INNER JOIN** - возвращает записи, имеющие общие значения в обеих таблицах.



- **LEFT OUTER JOIN** - возвращает строки со значениями, общими для таблиц, и все строки *левой* таблицы (т. е., даже те, для которых нет совпадений в правой таблице).
- **RIGHT OUTER JOIN** - возвращает строки со значениями, общими для таблиц, и все строки *правой* таблицы (т. е., даже те, для которых нет совпадений в левой таблице).
- **FULL OUTER JOIN** - возвращает все строки левой и правой таблиц.



Вопрос 17. Что такое нормализация и денормализация?



Ответ: **Нормализация** используется для уменьшения избыточности и зависимости данных. Достигается это путем организации полей и таблиц. Сюда входит создание таблиц и установка отношений между этими таблицами в соответствии с определенными правилами. С помощью этих правил можно избавиться от избыточности и противоречивых зависимостей, благодаря чему база данных становится более гибкой. **Денормализация** - это обратный процесс. В ходе денормализации мы добавляем избыточные данные. Это делается чтобы ускорить обработку сложных запросов, связанную с объединением данных из нескольких таблиц. При денормализации мы пытаемся оптимизировать производительность чтения базы данных, добавляя избыточные данные или группируя их.



Ссылки на доп.материал:

Видео про нормализацию данных (очень доступно!):

<https://www.youtube.com/watch?v=gDDrPL75LJY&list=LL&index=1>





Вопрос 18. Что такое индекс?



Ответ: Индексы помогают ускорить поиск в базе данных. Если в выражении WHERE не задан индекс никакого столбца, SQL-сервер в поиске соответствия будет проверять каждую строку во всей таблице. Если у вас много данных, эта операция может оказаться довольно медленной.

Индексы используются для поиска всех строк, совпадающих по каким-то столбцам, а затем уже в этих выборках ведется поиск нужных данных.

Синтаксис:

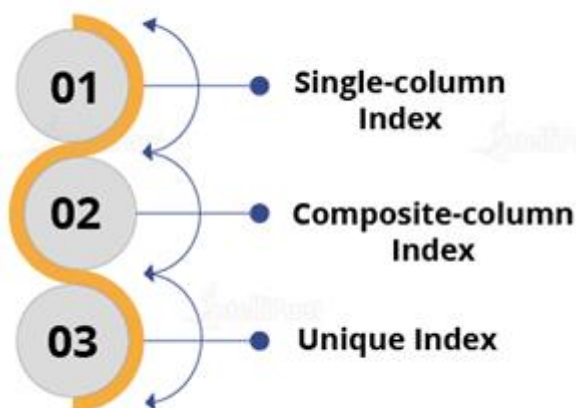
```
CREATE INDEX INDEX_NAME ON TABLE_NAME (COLUMN)
```



Вопрос 19. Расскажите о видах индексов.



Ответ:



Простые индексы. Создаются только для одного столбца таблицы.

Синтаксис:

```
CREATE INDEX index_name  
ON table_name(column_name);
```

Составные индексы. Создаются для двух или большего количества столбцов таблицы.



Синтаксис:

```
CREATE INDEX index_name
ON table_name (column1, column2)
```

Уникальные индексы. Используются для поддержания целостности данных таблицы. Они не дают вставлять в таблицу несколько значений.

Синтаксис:

```
CREATE UNIQUE INDEX index
ON table_name(column_name)
```



Вопрос 20. Что такое AUTO_INCREMENT?



Ответ: **AUTO_INCREMENT** используется в SQL для автоматической генерации уникального номера при каждом добавлении записи в таблицу.

Поскольку первичный ключ уникален для каждой записи, мы добавляем это поле в качестве **AUTO_INCREMENT** поля, таким образом при каждой вставке новой записи номер будет увеличиваться автоматически.

По умолчанию значение **AUTO-INCREMENT** начинается с 1 и увеличивается на 1 при каждом добавлении новой записи.

Синтаксис:

```
CREATE TABLE Employee (
  Employee_id int NOT NULL AUTO-INCREMENT,
  Employee_name varchar(255) NOT NULL,
  Employee_designation varchar(255)
  Age int,
  PRIMARY KEY (Employee_id)
)
```



ЧАСТЬ 3. ВОПРОСЫ ПО ИНТЕГРАЦИИ И АРХИТЕКТУРЕ.



Вопрос 21. Что такое API?



Ответ: API (Application Programming Interface — программный интерфейс приложения, или интерфейс программирования приложений) - это набор способов и правил, по которым различные программы общаются между собой и обмениваются данными. Главная цель использования API — связывание компонентов одного приложения с другим. Т.е. если API перестанет работать, то отключатся и все связанные с ним сервисы, инструменты, программы.

Наиболее распространенные виды или типы API:

- **RPC** (Remote Procedure Call) — удаленный вызов процедур
- **SOAP** (Simple Object Access Protocol) — простой протокол доступа к объектам
- **REST** (Representational State Transfer) — передача состояния представления
- **GraphQL** - маршрутизатор API-запросов внутри больших ИС и сложных связок сервисов



Ссылки на доп.материал:

Что такое API (простыми словами)

<https://superbwebsitebuilders.com/ru/chto-takoe-api-prostymi-slovami/>



Вопрос 22. В чем разница между SOAP и REST?



Ответ: Главное различие между REST и SOAP, это то, что REST — это архитектурный стиль. SOAP — это формат обмена сообщениями.

Специфика SOAP - это формат обмена данными. С SOAP это всегда SOAP-XML, который представляет собой XML, включающий: Envelope (конверт), Header (заголовок), Body (тело), Fault.

Специфика REST — использование HTTP в качестве транспортного протокола. Он подразумевает наилучшее использование функций, предоставляемых HTTP — методы запросов, заголовки запросов, ответы, заголовки ответов и т. д.



Ссылки на доп.материал:

REST vs SOAP

<https://otus.ru/nest/post/2427/>





Вопрос 23. Что такое архитектура программного обеспечения (ПО)?



Ответ: Архитектура программного обеспечения (ПО) - это структура и организация системы, которая определяет ее компоненты, взаимодействия между ними и их отношения с внешней средой. Также можно сказать, что Архитектура представляет собой основу для дальнейшей разработки ПО, определяет его функциональность, надежность, масштабируемость, производительность и возможности расширения. Важной частью архитектуры ПО является выбор технологий для ее реализации. Существует множество различных технологий, включая языки программирования, базы данных, фреймворки и библиотеки. Выбор технологий должен основываться на конкретных требованиях приложения, его цели и ограничениях.



Ссылки на доп.материал:

Разработка архитектуры для чайников. Часть 1.

<https://habr.com/ru/articles/658145/>

Разработка архитектуры для чайников. Часть 2.

<https://habr.com/ru/articles/658151/>



Вопрос 24. Что такое микросервисная, монолитная и клиент-сервисная архитектура? В чем разница между ними?



Ответ: **Микросервисная архитектура** - это подход, в котором приложение разделено на множество небольших сервисов, каждый из которых отвечает за определенную функцию. Она обеспечивает гибкость, относительную независимость компонентов и масштабируемость. Однако, такая архитектура может потребовать дополнительных усилий в сопровождении и управлении сервисами.

Монолитная архитектура - это крупнопланировочная структура с единым кодом, который включает в себя все компоненты приложения. Она обеспечивает простоту в разработке и тестировании, но часто имеет проблемы масштабирования и зависимости между компонентами.

Клиент-серверная архитектура - это модель, в которой клиенты на своих устройствах используют удаленный сервер для обмена информацией. Она обеспечивает хорошую масштабируемость и безопасность, но может иметь проблемы с производительностью из-за удаленного доступа.

Клиент-серверная и микросервисная архитектуры часто являются лучшим выбором для крупных проектов, которые требуют масштабируемости и гибкости. Монолитная архитектура хорошо подходит для небольших проектов или простых систем.





Ссылки на доп.материал:

Сравнение микросервисной и монолитной архитектур

<https://www.atlassian.com/ru/microservices/microservices-architecture/microservices-vs-monolith>

Микросервисы или монолит. Какую архитектуру выбрать при разработке сложного приложения для крупного бизнеса

<https://www.itweek.ru/management/article/detail.php?ID=223826>



Вопрос 25. Какие вы знаете виды и способы интеграции систем?



Ответ: Существует несколько видов интеграций систем, в том числе:

1. **Интеграция API** (Application Programming Interface) - это способ связывания различных программных приложений через их программные интерфейсы.
2. **Интеграция через файловые форматы** - это способ интеграции, основанный на конвертировании файлов в разные форматы для обмена информацией между ПО.
3. **Интеграция баз данных** - это способ обмена информацией между различными базами данных на основе протоколов обмена.
4. **Интеграция с помощью платформы** - это способ связывания различных приложений через одну платформу.

Способ интеграции зависит от требований конкретного проекта, но важно выбрать наиболее подходящий вариант для эффективной работы приложения в целом.



Ссылки на доп.материал:

Способы интеграции систем

<https://www.youtube.com/watch?v=Bw8f1Dd6Wr4>



ЧАСТЬ 4. ПРАКТИЧЕСКИЕ КЕЙСЫ.

1. Кейс 1. Проблемы с заказом комплектующих для выпуска конечного продукта.

Описание: к Вам пришел заказчик со следующей проблемой: постоянно происходит путаница в заказах комплектующих, из-за этого тормозим производство, и, следовательно, теряем клиентов. Какие вопросы Вы зададите заказчику и другим ответственным лицам?

Решение: Заказчику можно задать следующие вопросы:

- Что производит завод заказчика?
- В чем выражаются проблемы в заказах комплектующих (пересорт, например). Опишите один-два кейса, которые имели существенные негативные эффекты на бизнес.
- Кто отвечает за заказ комплектующих (человек с производства, который разбирается в деталях или, секретарь, например)?
- Каким образом формируется список того, что нужно заказать (в электронном виде подают/в системе через заявки на потребность, например /устно и т.д.)
- Как организован складской учет?

Вывод: вопросы будут строиться исходя из ответов заказчика☺

2. Кейс 2. У заказчика постоянно возникают новые идеи как доработать продукт.

Описание: к Вам постоянно приходит недовольный заказчик, который говорит, что все сделано не так, как нужно и требуется все переделать. Ваши действия?

Решение: получайте формальное согласование ТЗ. Идеально организовать встречу, куда пригласить заказчика, разработчиков, тестировщиков и вместе провалидировать требования.

Статья по теме есть здесь <https://vc.ru/hr/333579-chastye-oshibki-biznes-analitika-2>

3. Кейс 3. Бухгалтерия вносит данные в учетную систему в закрытом периоде.

Описание: Заказчик пришел со следующей проблемой – бухгалтерия периодически вносит данные в учетную систему в закрытом периоде, и это влияет на бонусы менеджеров по продажам (они рассчитываются ежеквартально). Бухгалтера игнорируют просьбы не трогать закрытый период, аргументируя, что это стандартная практика. Что делать в данной ситуации?

Решение: Бизнес-аналитик должен оценить риски и последствия от действий бухгалтерии, подготовить карту рисков и обсудить ее на встрече с руководителем бухгалтерии и менеджментом компании. По итогам данной встречи будет подготовлен регламент для бухгалтерии, позволяющий снизить бизнес-риски (некорректность расчета бонусов для менеджеров по продажам).

Риски хорошо рассмотрены здесь <https://practicum.yandex.ru/blog/riski-proekta-analiz-i-upravlenie/>.



ЧАСТЬ 5. ТЕСТОВЫЕ ЗАДАНИЯ.

Тестовое задание 1 (от банка из топ-10).

1. Сделать перечень БП для автоматизации деятельности городской библиотеки.
2. Сделать описание и схему одного из БП в библиотеке (из перечня п.1)
3. Сделать постановку на разработку одного из справочников для работы библиотеки.
4. Сделать перечень тест-кейсов для проверки реализации, один кейс описать подробно

Тестовое задание 2 (от девелопера)

Кейс 1 Процесс ловли зайца

Пётр профессионал в ловле зайцев на кирпич. Как только наступает 6 утра и Пётр видит, что на земле лежит снег, он проверяет, есть ли на заднем дворе красный кирпич. Если на заднем дворе нет красного кирпича, то Пётр одалживает красный кирпич у своего соседа Андрея. После получения красного кирпича Пётр проверяет, есть ли на верхней полке чёрный молотый перец. Если перца нет, то Пётр ждёт час и проверяет наличие перца ещё раз. Когда Пётр находит перец на верхней полке (жена Петра Зинаида его туда перекладывает с нижней полки, как только замечает, что её муж вошёл в цикл), Пётр берёт молотый перец и красный кирпич и идёт в поле. Там он кладёт кирпич на снег, посыпают его перцем и прячется. Заяц видит красный кирпич на снегу, думает, что это морковь, подбегает к кирпичу, нюхает его, чихает, ударяется головой об кирпич и теряет сознание. Тогда Пётр выходит из засады и ловит беспомощного зайца. Либо заяц может не видеть кирпич достаточно долго. Пётр ждёт 8 часов и если заяц не появляется, то Пётр идёт домой.

Кейс 2 Система управления заявками

Система управления заявками (далее - Система) должна представлять собой веб-приложение, реализованное на основании REST-API. Для хранения данных должна использоваться реляционная БД.

Команде разработчиков необходимо реализовать в Системе механизм работы с заявками, который должен включать возможности:

- регистрация заявки;
- просмотр данных по заявке;
- редактирование заявки;
- работа со списком заявок.

Процесс работы с заявками состоит из шагов:

- регистрация заявки. Варианты выхода: обработка заявки;
- обработка заявки. Варианты выхода: создание дела / отказ в создании дела.

В форме регистрации заявки должна быть реализована возможность ввода данных:



- • номер заявки - заполняется вручную;
- • дата регистрации - выбор даты в календаре;
- • источник получения - выбор одного значения из справочника;
- • вид работ - выбор одного значения из справочника;
- • заказчик - выбор одного значения из списка заказчиков;
- • название объекта - заполняется вручную;
- • адрес объекта - заполняется вручную;
- • тип объекта - выбор одного значения из справочника.

В форме обработки заявки должна быть реализована возможность ввода данных:

- • номер заявки - заполняется вручную;

Задание

1. Постройте модель BPMN процесса ловли зайца по кейсу 1.
2. Разработайте «Регламент ловли зайца» по кейсу 1.
3. Презентуйте основному заинтересованному лицу не менее двух изменений процесса ловли зайца по кейсу 1.
4. Разработайте ER-модель Системы по кейсу 2.



P.S. от @AnakondaMar

В этом конспекте постаралась собрать все наиболее частые вопросы к собеседованиям с ответами и с доп.материалами. Очень надеюсь, что мои материалы были Вам полезны! Желаю Вам твердо и уверенно ворваться в профессию БА и заявить о себе! 😊

P.S. Данная версия методички исключительно для Вас. Прошу не делиться ей с другими студентами Skillbox и своими знакомыми! Просьба направлять их ко мне @AnakondaMar. Спасибо за понимание с Вашей стороны 😊

